[redacted]

Important: Text generated by AI and may contain incorrect information, always check the content.

To identify outliers in the data, I'll first need to check the distribution of the numerical columns. The numerical columns in the DataFrame are 'label', 'confidence', and 'social_timestamp'.

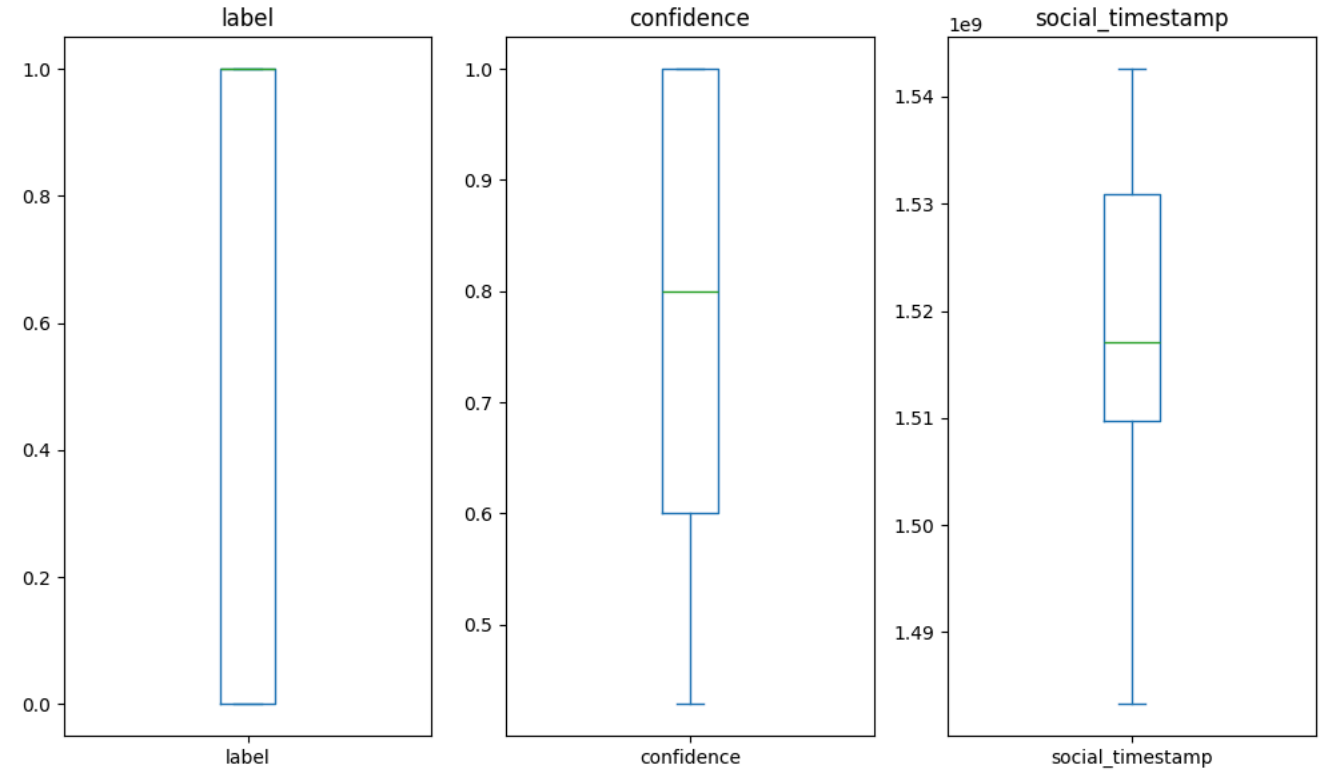
```python
import pandas as pd
import matplotlib.pyplot as plt

# Check the distribution of numerical columns
numerical_columns = ['label', 'confidence', 'social_timestamp']
display(df[numerical_columns].describe())

# Boxplot to visualize the distribution
plt.figure(figsize=(10,6))
for i, col in enumerate(numerical_columns):
    plt.subplot(1, 3, i+1)
    df[col].plot.box()
    plt.title(col)
plt.tight_layout()
plt.show()
```

	label	confidence	social_timestamp
count	2838.000000	2838.000000	2.838000e+03
mean	0.524313	0.808972	1.518107e+09
std	0.499497	0.177038	1.552209e+07
min	0.000000	0.428571	1.483274e+09
25%	0.000000	0.600000	1.509698e+09
50%	1.000000	0.800000	1.517066e+09
75%	1.000000	1.000000	1.530898e+09
max	1.000000	1.000000	1.542592e+09

[redacted]



Distribution of 'label' and 'confidence' indicate that there are no significant outliers, as the values are mostly concentrated around the median. However, 'confidence' has a minimum value of 0.428571429

For 'social_timestamp', the boxplot may not be the best representation due to the large range of values. Let's check the distribution using a different method.

```python
# Check for outliers in 'social_timestamp'
Q1 = df['social_timestamp'].quantile(0.25)
Q3 = df['social_timestamp'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
display(df[(df['social_timestamp'] < lower_bound) | (df['social_timestamp'] > upper_bound)])
```

subreddit	post_id	sentence_range	text	label	confidence	social_timestamp

This will help identify any outliers in the 'social_timestamp' column.

Also, we can check the 'social_timestamp' column for any unusual patterns or values.

```python
# Check the 'social_timestamp' column
display(df['social_timestamp'].min())
display(df['social_timestamp'].max())
display(pd.to_datetime(df['social_timestamp'], unit='s').min())
display(pd.to_datetime(df['social_timestamp'], unit='s').max())
```

```
1483274218
1542592460
Timestamp('2017-01-01 12:36:58')
```

[redacted]

Timestamp('2018-11-19 01:54:20')
Let's convert 'social_timestamp' to datetime to better understand the range of dates.

```python
# Convert 'social_timestamp' to datetime
df['date'] = pd.to_datetime(df['social_timestamp'], unit='s')
display(df['date'].describe())
```

<string>:3: FutureWarning: Treating datetime data as categorical rather than numeric in `.describe` is deprecated and will be removed in a future version of pandas. Specify `datetime_is_numeric=True` to

	date
count	2838
unique	2343
top	2018-10-08 08:49:45
freq	6
first	2017-01-01 12:36:58
last	2018-11-19 01:54:20

dtype: object

[redacted]